
Football Prediction Dashboard

ZERRARI ZIYAD
HAJJAD IKRAME
ABOUJJID MAYSSAE
ZIANI MOHAMED AMINE

Ingénierie Informatique et Réseaux
(3IIR)

2025 - 2026

Tuteur(s) EMSI :
PR. WAFAE ELKAYSOUNI

Membres du jury :
PR. WAFAE ELKAYSOUNI

Résumé

Dans l'ère du Big Data, l'analyse sportive (*Sports Analytics*) est devenue un domaine de recherche majeur. Ce projet vise à concevoir et développer une application web intelligente capable de prédire l'issue de matchs de football à partir de données historiques et tactiques. L'architecture repose sur une séparation claire entre un backend robuste (Django), un moteur d'intelligence artificielle (XGBoost) optimisé avec SMOTE pour gérer le déséquilibre des classes, et un frontend interactif et réactif (React). Ce document détaille l'intégralité du cycle de vie du projet : de l'analyse des besoins à la conception architecturale, en passant par les fondements mathématiques du Machine Learning et l'implémentation technique d'un système hautement sécurisé par JWT.

Mots-clés : Machine Learning, XGBoost, Django, React, Big Data, Football Analytics, SMOTE, JWT.

Abstract

In the era of Big Data, sports analytics has emerged as a major research field. This project aims to design and develop an intelligent web application capable of predicting the outcome of football matches using historical and tactical data. The architecture is based on a clear separation between a robust backend (Django), an artificial intelligence engine (XGBoost) optimized with SMOTE to handle class imbalance, and an interactive and responsive frontend (React). This document details the entire lifecycle of the project : from requirements analysis to architectural design, through the mathematical foundations of Machine Learning and the technical implementation of a highly secure JWT-based system.

Keywords : Machine Learning, XGBoost, Django, React, Big Data, Football Analytics, SMOTE, JWT.

Remerciements

Nous tenons à exprimer nos sincères remerciements à notre encadrante pédagogique, **Pr. Wafae Elkaysouni**, pour son encadrement, ses conseils précieux et sa disponibilité tout au long de la réalisation de ce projet. Ses remarques pertinentes et son suivi rigoureux ont grandement contribué à l'amélioration de la qualité de ce travail. Nous adressons également nos remerciements à nos collègues pour leur soutien, leur esprit de collaboration et les échanges enrichissants que nous avons partagés.

Table des matières

Résumé	1
Abstract	2
Remerciements	3
Introduction générale	8
1 Analyse et Étude Préalable	10
1.1 État de l'art	10
1.2 Cahier des charges fonctionnel	11
1.2.1 Fonctionnalités pour l'Utilisateur Standard	11
1.2.2 Fonctionnalités pour l'Administrateur	12
1.3 Exigences Non-Fonctionnelles	12
1.4 Analyse des Risques	12
1.5 Modélisation des Processus (UML)	12
1.5.1 Diagramme de Cas d'Utilisation	12
1.5.2 Spécification détaillée des cas d'utilisation	14
Conclusion du chapitre	15
2 Choix Technologiques et Modélisation	16
2.1 Environnement Technique	16
2.1.1 Backend : Django & REST Framework	16
2.1.2 Frontend : React & Zustand	17
2.1.3 Fournisseur de Données : API-Football	17
2.2 Fondements Théoriques de l'Intelligence Artificielle	17
2.2.1 Le Problème de Déséquilibre des Classes (<i>Imbalanced Data</i>)	17
2.2.2 Algorithme Prédicatif : XGBoost	18
2.3 Ingénierie des Caractéristiques (<i>Feature Engineering</i>)	19
2.4 Évaluation et Validation du Modèle	19
Conclusion du chapitre	19

3	Conception Architecturale	20
3.1	Architecture N-Tiers (<i>Backend as a Service</i>)	20
3.2	Diagrammes de Séquence	20
3.2.1	Processus d'Authentification	21
3.3	Catalogue des API REST	21
3.4	Modélisation de la Base de Données	22
3.4.1	Diagramme de Classes (Backend)	22
3.4.2	Justification des choix de modélisation	22
	Conclusion du chapitre	23
4	Mise en œuvre : Backend, Frontend et Validation	24
4.1	Implémentation Backend et Pipeline de Données	24
4.1.1	Sécurisation et Authentification JWT	24
4.1.2	Orchestration du Pipeline de Données	25
4.1.3	Le Endpoint de Prédiction	26
4.2	Implémentation Frontend et UI/UX	26
4.2.1	Choix Technologiques et Architecture	26
4.2.2	Gestion de l'État (<i>State Management</i>)	26
4.2.3	Conception Visuelle (UI/UX)	27
4.3	Tests, Validation et Résultats	29
4.3.1	Évaluation du Modèle d'Intelligence Artificielle	29
4.3.2	Validation du Backend (API REST)	30
4.3.3	Validation du Frontend et de l'Ergonomie	31
4.4	Limites du système	31
	Conclusion du chapitre	32
	Conclusion générale	33

Table des figures

1.1	Diagramme de Cas d'Utilisation du système complet	13
2.1	Framework Backend Django	16
2.2	Bibliothèque Frontend React	17
2.3	Fournisseur de données sportives	17
3.1	Architecture technique à N-Tiers	20
3.2	Diagramme de Séquence : Flux d'Authentification JWT	21
3.3	Modèle conceptuel de données simplifié de l'application (ORM Django)	22
4.1	Aperçu de l'interface du Dashboard analytique	28
4.2	Aperçu de l'interface de Connexion et Sécurité	29

Liste des tableaux

1.1	Tableau comparatif des modèles d'apprentissage automatique pour la prédiction sportive	11
1.2	Description textuelle du cas d'utilisation « Consulter les prédictions »	14
1.3	Description textuelle du cas d'utilisation « Synchroniser les données API »	14
3.1	Catalogue des principaux endpoints de l'API REST	21
4.1	Structure de la Matrice de Confusion pour l'évaluation du modèle . .	30
4.2	Résultats des cas de tests d'intégration de l'API Backend	31

Introduction générale

L'industrie du sport a connu une métamorphose radicale au cours de la dernière décennie. Avec l'avènement du Big Data et des capacités de calcul à grande échelle, la prise de décision empirique a laissé place à une approche scientifique et mathématique. Dans le football, sport caractérisé par un haut niveau d'incertitude et une rareté des buts (*faible scoring rate*), anticiper l'issue d'une rencontre est un défi de taille qui fascine autant les chercheurs en intelligence artificielle que les analystes sportifs.

Contexte et Enjeux

Traditionnellement, la prédiction sportive reposait sur l'expertise humaine, souvent biaisée par des facteurs émotionnels ou des observations subjectives. Aujourd'hui, grâce aux fournisseurs de données comme API-Football, il est possible de quantifier chaque aspect d'une rencontre : pourcentage de possession, tirs cadrés, précision des passes, etc. L'enjeu majeur est de transformer ce volume massif de données brutes en un signal prédictif exploitable, tout en contournant les pièges statistiques liés au football (notamment la difficulté à prédire un match nul).

Objectifs du Projet

L'objectif principal de ce projet de fin d'année est de concevoir, développer et déployer une plateforme complète nommée **Intelligent Football Prediction Dashboard**. Ce système doit accomplir les tâches suivantes :

1. **Ingestion automatisée** : Collecter et stocker massivement les données historiques et tactiques des ligues de football.
2. **Modélisation prédictive** : Développer un modèle de Machine Learning robuste capable de calculer les probabilités de victoire (Home, Draw, Away).
3. **Restitution visuelle** : Offrir une interface utilisateur (UI) moderne et ergonomique pour l'analyse décisionnelle.
4. **Sécurisation** : Garantir l'intégrité des données et des accès via un système d'authentification par rôles.

Structure du Rapport

Afin de relater notre démarche d'ingénierie, ce rapport s'articule autour de quatre chapitres :

- Le **Chapitre 1** présente l'analyse fonctionnelle, le cahier des charges et l'état de l'art.
- Le **Chapitre 2** expose les fondements théoriques des algorithmes de Machine Learning utilisés (XGBoost, SMOTE).
- Le **Chapitre 3** détaille la conception architecturale (UML, Base de données).
- Enfin, le **Chapitre 4** couvre l'intégralité de la mise en œuvre technique : du développement Backend à l'interface Frontend, en passant par la phase exhaustive de tests et de validation.

Chapitre 1

Analyse et Étude Préalable

Ce premier chapitre a pour objectif de poser les bases du projet. Il définit le cadre du travail, décrit les besoins des utilisateurs et définit les caractéristiques techniques et fonctionnelles.

1.1 État de l’art

L’application de l’intelligence artificielle au football n’est pas nouvelle, mais ses méthodes ont beaucoup évolué.

Historiquement, les modèles de prédiction utilisaient la loi de Poisson pour calculer la distribution des buts [1]. Bien qu’efficaces, ces modèles ont un problème majeur : ils supposent que les buts marqués par chaque équipe sont des événements indépendants, ce qui est contredit par la réalité d’un match (un but change la stratégie adverse). Dixon et Coles [2] ont proposé une correction de cette indépendance pour les petits scores (0-0, 1-0, etc.), améliorant les prédictions.

Récemment, l’utilisation des méthodes d’apprentissage automatique a transformé ce domaine, permettant d’utiliser des variables complexes (fatigue, forme récente, expected goals). Afin d’expliquer notre choix technique, le tableau 1.1 présente un résumé des différents modèles de classification habituellement utilisés dans les études existantes [3].

Modèle	Int.	Avantages	Limites	Réf.
Régression Logistique	Haute	Rapide, simple, probabilités claires.	Difficulté avec les relations complexes.	[3]
SVM	Faible	Efficace pour les données complexes.	Sensible à l'échelle des données.	[4]
Random Forest	Moyenne	Modèle robuste, réduit le surapprentissage.	Aspect "boîte noire", lourd en mémoire.	[5]
XGBoost	Moyenne	Correction des erreurs, très haute précision.	Risque de sur-apprentissage si mal réglé.	[6]

TABLE 1.1 – Tableau comparatif des modèles d'apprentissage automatique pour la prédiction sportive

Face à ces alternatives, notre choix s'est porté sur une méthode plus performante pour les données tabulaires : le **Gradient Boosting**, et plus particulièrement son implémentation **XGBoost** [6]. Contrairement au Random Forest qui construit ses arbres séparément, XGBoost les construit l'un après l'autre, chaque nouvel arbre corrigeant les erreurs des précédents. Cette approche offre une grande souplesse et des performances souvent supérieures aux modèles traditionnels.

1.2 Cahier des charges fonctionnel

L'application doit proposer un ensemble de fonctionnalités selon le type d'utilisateur.

1.2.1 Fonctionnalités pour l'Utilisateur Standard

- **Consultation des matchs** : Voir le calendrier des matchs à venir et les résultats récents.
- **Analyse des compositions** : Consulter les formations (ex : 4-3-3), les titulaires et les remplaçants.
- **Demande de prédiction** : Obtenir l'estimation de l'IA (pourcentages de chances) pour un match précis.
- **Comparatif tactique** : Comparer visuellement (graphiques Radar) la forme et les statistiques des deux équipes.

1.2.2 Fonctionnalités pour l'Administrateur

- **Gestion des accès** : Voir les utilisateurs inscrits sur la plateforme.
- **Mise à jour des données** : Lancer manuellement la récupération des données depuis l'API-Football (Matches, Statistiques, Compositions).

1.3 Exigences Non-Fonctionnelles

En plus des fonctionnalités de base, le système doit répondre à des critères de qualité précis :

- **Sécurité** : L'accès doit être protégé par une connexion sécurisée (JWT). Les mots de passe sont cryptés en base de données.
- **Qualité des données** : Le système garantit l'unicité des matchs importés via l'identifiant `fixture_id` pour éviter les doublons.
- **Rapidité** : Le moteur d'IA doit fournir une réponse en moins de 0,5 seconde une fois les données chargées.

1.4 Analyse des Risques

Le projet identifie deux risques majeurs liés aux données externes :

- **Limitation des API (Rate Limiting)** : L'API gratuite limite le nombre de requêtes par jour. Pour résoudre cela, nous enregistrons les données localement (SQLite) pour ne pas interroger l'API plusieurs fois pour le même match.
- **Qualité des données** : Certaines compositions peuvent être incomplètes. Le système a été conçu pour être souple et traiter ces cas sans bloquer l'utilisateur.

1.5 Modélisation des Processus (UML)

1.5.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation permet d'identifier les acteurs du système et leurs interactions directes avec les fonctionnalités offertes.

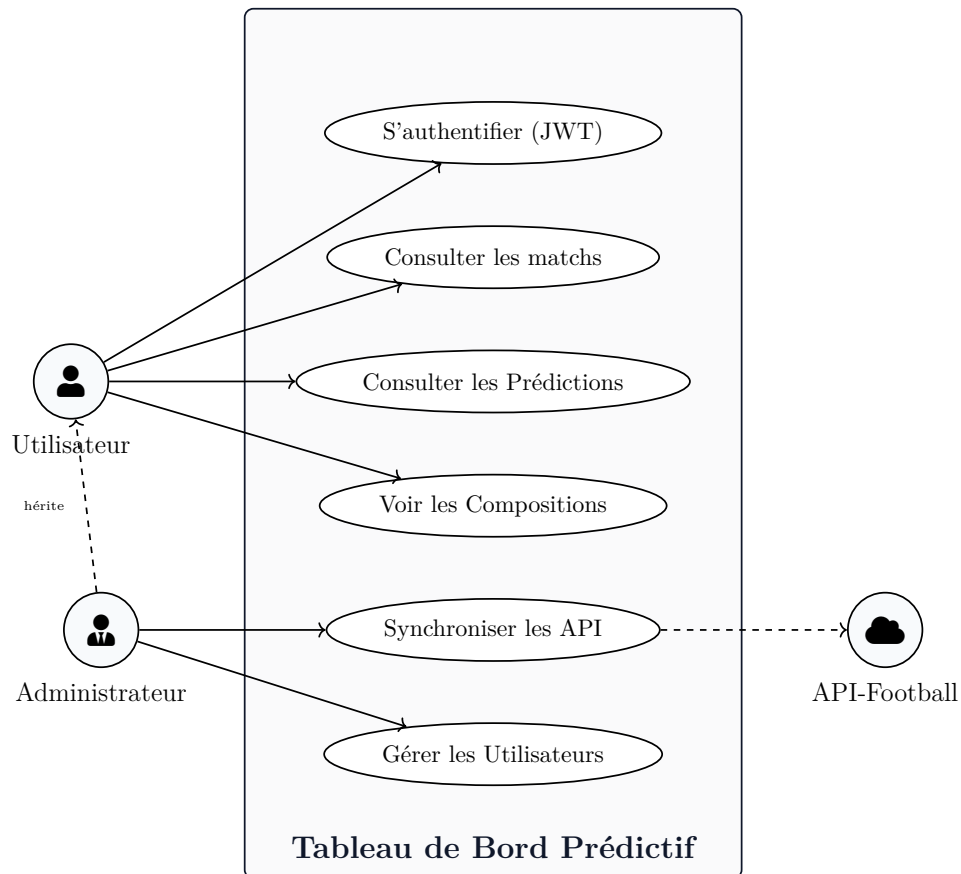


FIGURE 1.1 – Diagramme de Cas d'Utilisation du système complet

1.5.2 Spécification détaillée des cas d'utilisation

CU-01 — Consulter les prédictions

Acteur principal	Utilisateur, Administrateur
Précondition	L'utilisateur doit être authentifié sur la plateforme.
Scénario nominal	L'utilisateur accède à l'onglet <i>Predictions</i> , le système affiche la liste des matchs à venir, puis il sélectionne un match. Le frontend envoie ensuite une requête API au backend, qui interroge le modèle XGBoost à partir des données du match. Le système affiche enfin les probabilités de résultat ainsi que le radar tactique.
Post-condition	Les statistiques prédictives du match sélectionné sont affichées à l'écran.

TABLE 1.2 – Description textuelle du cas d'utilisation « Consulter les prédictions »

CU-02 — Synchroniser les données API

Acteur principal	Administrateur
Précondition	L'utilisateur doit être authentifié et disposer du rôle ADMIN.
Scénario nominal	L'administrateur accède au panneau d'administration, clique sur le bouton <i>Fetch Latest Fixtures</i> , puis le système lance un processus d'arrière-plan. Le serveur récupère ensuite les données au format JSON depuis API-Football et met à jour la base de données SQLite locale.
Exception	En cas de dépassement de la limite de requêtes API, le système interrompt le processus proprement et affiche un message d'erreur.

TABLE 1.3 – Description textuelle du cas d'utilisation « Synchroniser les données API »

Conclusion du chapitre

Ce chapitre a permis de définir le cadre de notre projet et d'identifier les besoins essentiels des utilisateurs. L'étude de l'état de l'art a justifié le choix de l'algorithme XGBoost, tandis que la modélisation UML a clarifié les interactions au sein du système. Ces bases solides nous permettent maintenant d'aborder les choix technologiques et la modélisation mathématique du moteur de prédiction.

Chapitre 2

Choix Technologiques et Modélisation

Ce chapitre présente d’abord les technologies clés sélectionnées pour développer notre architecture logicielle, puis détaille le fonctionnement de notre moteur d’intelligence artificielle.

2.1 Environnement Technique

Pour garantir la performance et la maintenance du projet, nous avons choisi une séparation nette entre le *frontend* et le *backend*.

2.1.1 Backend : Django & REST Framework



FIGURE 2.1 – Framework Backend Django

Le serveur est développé en Python avec le framework Django, complété par **Django REST Framework (DRF)**. Ce choix est expliqué par la puissance des *Serializers* de DRF, qui permettent de transformer les modèles complexes de notre base de données (incluant les statistiques et compositions) en flux JSON utilisables par le frontend. Il assure également la sécurité des requêtes via JWT et la gestion du modèle XGBoost.

2.1.2 Frontend : React & Zustand

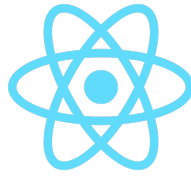


FIGURE 2.2 – Bibliothèque Frontend React

L'interface utilisateur est construite avec React. Pour la gestion de l'état global (connexion, liste des matchs, résultats de l'IA), nous avons utilisé **Zustand**. Contrairement à Redux, Zustand est plus simple et rapide, ce qui réduit la complexité du code tout en garantissant une mise à jour fluide des composants de l'application.

2.1.3 Fournisseur de Données : API-Football



FIGURE 2.3 – Fournisseur de données sportives

La source de données de notre modèle de prédiction provient d'API-Football. Elle nous fournit des données historiques complètes et actualisées (résultats, compositions, statistiques de match) via des requêtes REST.

2.2 Fondements Théoriques de l'Intelligence Artificielle

2.2.1 Le Problème de Déséquilibre des Classes (*Imbalanced Data*)

Dans le football de haut niveau, les résultats ne sont pas répartis de manière égale. Les victoires à domicile (*Home Win*) représentent environ 45% des cas, les victoires à l'extérieur (*Away Win*) environ 30%, et les matchs nuls (*Draw*) environ 25%. Si nous entraînons un algorithme sur ces données sans modification, il aura tendance à privilégier l'équipe à domicile et ne réussira pas à prédire le match nul, car c'est le cas le moins fréquent.

Approche Mathématique : SMOTE

Pour résoudre ce problème, nous avons utilisé **SMOTE** (*Synthetic Minority Over-sampling Technique*) [7]. Contrairement à la simple copie de données, SMOTE crée des exemples synthétiques entre les points existants de la classe minoritaire.

Soit un point $x_i \in$ Classe Minoritaire. On sélectionne l'un de ses k plus proches voisins, x_{zi} . Le nouvel échantillon synthétique x_{syn} est généré par la formule :

$$x_{syn} = x_i + \lambda \times (x_{zi} - x_i) \quad (2.1)$$

où λ est un nombre aléatoire entre 0 et 1. Cette technique permet d'enrichir les données d'entraînement et force le modèle à mieux apprendre les caractéristiques du "Match Nul".

2.2.2 Algorithme Prédicatif : XGBoost

Nous avons choisi *Extreme Gradient Boosting* (XGBoost), un algorithme basé sur les arbres de décision [6]. Il utilise la technique du *Boosting* où chaque nouvel arbre est construit pour corriger les erreurs des arbres précédents.

Fonction d'Objectif et Régularisation

XGBoost cherche à minimiser une fonction d'objectif composée d'une fonction de perte (*Loss*) et d'un terme de régularisation (pour éviter de trop coller aux données d'entraînement) :

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (2.2)$$

Avec le terme de régularisation défini par :

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (2.3)$$

où T est le nombre de feuilles de l'arbre, et w_j le poids de la feuille j .

Adaptation à la classification multiclasse

Puisque notre problème compte 3 issues possibles (H, D, A), le modèle utilise la fonction objectif `multi:softprob`. Il applique une fonction Softmax sur les scores de sortie pour générer une distribution de probabilité :

$$P(y = k | x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (2.4)$$

C'est cette probabilité (ex : Home 45%, Draw 20%, Away 35%) qui est renvoyée au frontend React pour afficher le graphique.

2.3 Ingénierie des Caractéristiques (*Feature Engineering*)

L'algorithme utilise un vecteur de 25 caractéristiques. Pour assurer le bon fonctionnement du modèle, toutes les variables numériques sont normalisées avec **StandardScaler** :

$$z = \frac{x - \mu}{\sigma} \quad (2.5)$$

où μ est la moyenne et σ l'écart-type. Parmi les variables les plus importantes :

- **Form Points** : Points obtenus lors des 5 derniers matchs (3 pour une victoire, 1 pour un nul).
- **Goal Ratios** : Moyenne de buts marqués vs encaissés sur la saison.
- **Home Advantage** : Variable représentant l'avantage de jouer à domicile.

2.4 Évaluation et Validation du Modèle

Pour garantir la fiabilité de nos prédictions, le système compare toujours le *Résultat Prédit* au *Résultat Réel* une fois le match terminé.

- **Accuracy Score** : Pourcentage total de bonnes prédictions.
- **Matrice de Confusion** : Analyse des erreurs pour voir quels types de résultats sont les plus difficiles à prédire.

Ces indicateurs sont calculés par le backend et affichés sur le tableau de bord de l'administrateur pour suivre la performance de l'IA.

Conclusion du chapitre

En résumé, ce chapitre a détaillé l'environnement technique reposant sur Django et React, ainsi que les piliers mathématiques de notre solution. L'utilisation de SMOTE pour équilibrer les données et de XGBoost pour la prédiction constitue le cœur intelligent de l'application. La maîtrise de ces outils et concepts est indispensable pour garantir la précision des résultats que nous allons maintenant structurer architecturalement.

Chapitre 3

Conception Architecturale

Une conception sérieuse est nécessaire pour un bon développement. L'application utilise une architecture séparée afin de bien diviser les responsabilités.

3.1 Architecture N-Tiers (*Backend as a Service*)

L'application est divisée en trois parties distinctes, qui communiquent via des API standardisées.

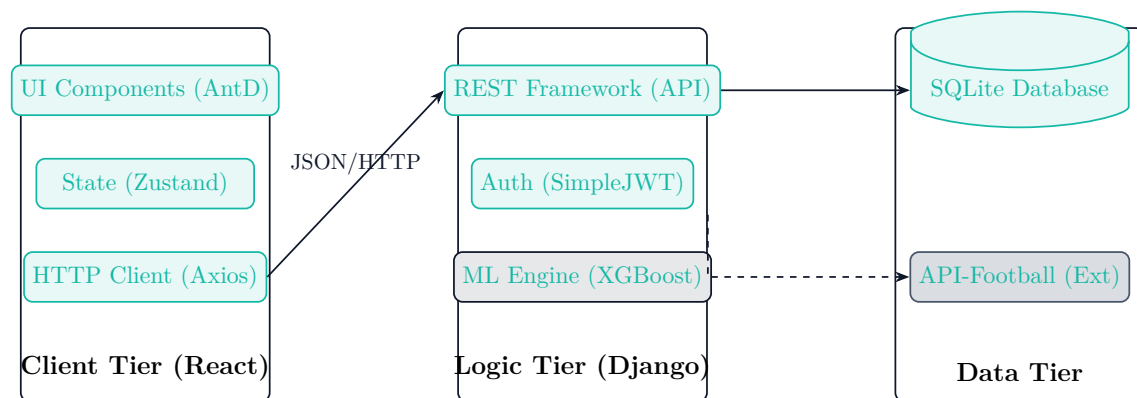


FIGURE 3.1 – Architecture technique à N-Tiers

3.2 Diagrammes de Séquence

Pour bien comprendre les interactions entre les différentes parties de notre application, nous avons décrit les étapes clés sous forme de diagrammes de séquence.

3.2.1 Processus d'Authentification

Le diagramme ci-dessous montre le flux de connexion basé sur les jetons JWT. Ce système permet de garantir que seules les personnes autorisées peuvent accéder aux prédictions.

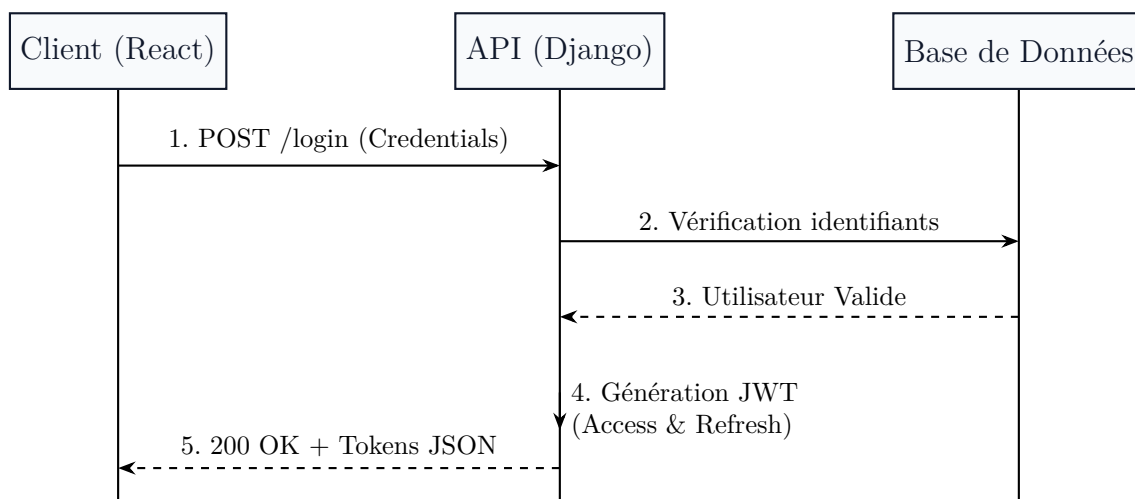


FIGURE 3.2 – Diagramme de Séquence : Flux d'Authentification JWT

3.3 Catalogue des API REST

La communication entre le frontend React et le backend Django est assurée par une API REST. Le tableau suivant présente les points d'entrée (endpoints) principaux.

Endpoint	Méthode	Description
/api/users/login/	POST	Connexion et génération du token JWT.
/api/matches/	GET	Liste des matchs disponibles.
/api/matches/{id}/	GET	Détails d'un match (stats, compositions, cotes).
/api/predictions/predict/	POST	Envoi d'un match au moteur XG-Boost.
/api/predictions/accuracy/	GET	Récupération des scores de performance.

TABLE 3.1 – Catalogue des principaux endpoints de l'API REST

3.4 Modélisation de la Base de Données

Le backend Django utilise une structure de données précise pour garantir la cohérence des informations sportives.

3.4.1 Diagramme de Classes (Backend)

Le diagramme suivant présente les éléments principaux gérant la structure d'un match de football dans notre base de données.

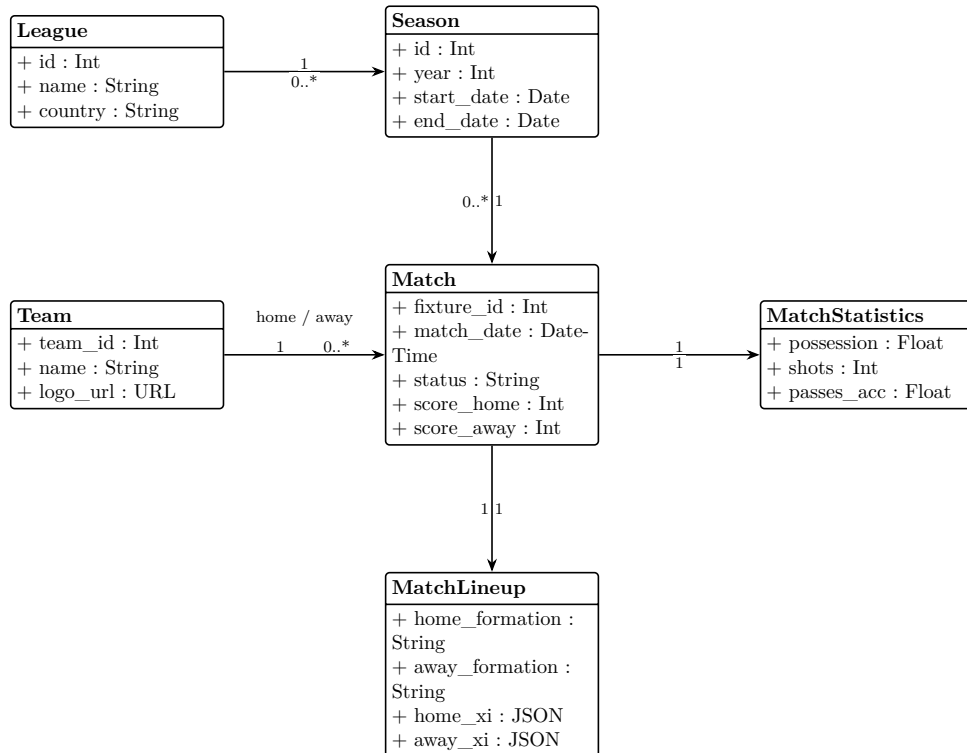


FIGURE 3.3 – Modèle conceptuel de données simplifié de l'application (ORM Django)

3.4.2 Justification des choix de modélisation

- **Relations One-to-One** : Les tables **MatchStatistics**, **MatchLineup** et **MatchOdds** sont liées au **Match** par des relations 1 :1. Ce découpage permet une meilleure organisation : nous pouvons charger un match sans alourdir la demande avec des statistiques si elles ne sont pas utiles.
- **Stockage JSON (JSONField)** : Le choix de stocker les compositions (**MatchLineup**) sous forme de **JSONField** est un choix technique pour la rapidité et la souplesse. Plutôt que de créer des milliers de lignes pour chaque

joueur, nous enregistrons l'objet "Composition" tel qu'il arrive de l'API. Cela facilite l'affichage côté React.

Conclusion du chapitre

Ce chapitre a présenté l'architecture globale du système, mettant en avant une séparation nette entre les services. La documentation de l'API REST et la modélisation optimisée de la base de données assurent une communication fluide et une gestion efficace des volumes de données sportives. Cette structure architecturale robuste sert de guide pour la phase de mise en œuvre technique détaillée dans le chapitre suivant.

Chapitre 4

Mise en œuvre : Backend, Frontend et Validation

Ce chapitre aborde la mise en œuvre technique de l'application dans son ensemble. Il détaille d'abord le développement côté serveur (Backend), puis la matérialisation de l'interface utilisateur (Frontend), pour se conclure par une phase exhaustive de tests et de validation.

4.1 Implémentation Backend et Pipeline de Données

Le backend est développé en **Python** avec le framework **Django**, réputé pour sa robustesse et sa sécurité native (*Out-of-the-box*).

4.1.1 Sécurisation et Authentification JWT

L'authentification traditionnelle par session n'est pas adaptée aux applications *Single-Page* (SPA) modernes. Nous avons implémenté le standard JWT (*JSON Web Token*) [8].

Déroulement de l'Authentification

Lorsqu'un utilisateur se connecte, le serveur génère deux tokens chiffrés :

1. **Access Token** : Durée de vie courte (60 minutes). Il contient l'identifiant de l'utilisateur et son rôle.
2. **Refresh Token** : Durée de vie longue (plusieurs jours). Il permet de générer un nouvel Access Token sans obliger l'utilisateur à se reconnecter.

Séparation des rôles (Admin/User)

Pour respecter le principe de préservation de la base de données existante, la notion de rôle n'a pas été codée en remplaçant le modèle utilisateur natif de Django, mais en lui liant un `UserProfile` :

```

1 from django.db import models
2 from django.contrib.auth.models import User
3 from django.db.models.signals import post_save
4 from django.dispatch import receiver
5
6 class UserProfile(models.Model):
7     ADMIN = 'ADMIN'
8     USER = 'USER'
9     ROLE_CHOICES = [(ADMIN, 'Admin'), (USER, 'User')]
10
11     user = models.OneToOneField(User, on_delete=models.CASCADE,
12                               related_name='profile')
13     role = models.CharField(max_length=10, choices=ROLE_CHOICES,
14                             default=USER)
15
16 # Signal automatisant la creation du profil
17 @receiver(post_save, sender=User)
18 def create_user_profile(sender, instance, created, **kwargs):
19     if created:
20         UserProfile.objects.create(user=instance)

```

Listing 4.1 – Création du profil et des signaux (users/models.py)

La protection des routes sensibles est assurée par une permission personnalisée `IsAdminRole` qui vérifie la propriété `role` avant d'autoriser l'exécution de la vue.

4.1.2 Orchestration du Pipeline de Données

La récolte des données depuis l'API-Football externe doit être soigneusement gérée pour ne pas dépasser les limites de requêtes (*Rate Limits*) imposées par le fournisseur.

Nous avons développé des **Commandes de Management Django** exécutables via le terminal ou déclenchables par l'interface d'administration. Le script ci-dessous montre la logique de protection implémentée lors de la récupération des compositions d'équipes (`fetch_match_lineups.py`) :

```

1 matches_to_fetch = Match.objects.filter(
2     status="FT",
3     lineup__isnull=True,
4     match_date__lte=tz.now()
5 ).order_by('-match_date')[:20] # BATCH: Max 20 requetes par
6     execution
7
8 for match in matches_to_fetch:

```

```
8     response = requests.get(f"{base_url}/fixtures/lineups", headers
9         =headers, params={'fixture': match.fixture_id})
10    data = response.json()
11
12    # Verification Critique : Rate Limit
13    if data.get('errors') and 'requests' in data.get('errors'):
14        self.stderr.write("Limite quotidienne atteinte. Arret de
15            securite.")
16        break # Protection de la cle API
17
18    time.sleep(6.2) # Delai obligatoire de 6 secondes entre chaque
19        requete
```

Listing 4.2 – Logique de Batching et Rate Limiting

4.1.3 Le Endpoint de Prédiction

Le module `predictions/views.py` charge en mémoire RAM le modèle XGBoost préalablement entraîné (`.pkl`) ainsi que son outil de normalisation (`StandardScaler`). À la volée, le serveur récupère les 25 statistiques du match demandé, les normalise, et interroge le modèle. L'API REST expose ensuite un point de terminaison sécurisé retournant le résultat sous un format JSON exploitable par le client.

4.2 Implémentation Frontend et UI/UX

L'interface utilisateur est le point d'interaction critique. Elle doit retranscrire la complexité mathématique des algorithmes en visuels clairs et actionnables.

4.2.1 Choix Technologiques et Architecture

Le frontend est développé en **React JS**, compilé avec **Vite** pour des performances optimales. L'écosystème s'appuie sur :

- **Ant Design (AntD)** : Bibliothèque de composants UI fournissant des grilles, des cartes et des tableaux hautement personnalisables.
- **Recharts** : Bibliothèque D3.js pour React, utilisée pour le rendu du Radar tactique et du graphique probabiliste.
- **Zustand** : Outil de gestion d'état global (*State Management*), plus léger et performant que Redux.

4.2.2 Gestion de l'État (*State Management*)

La communication avec le backend et la sauvegarde des tokens de sécurité sont centralisées dans le Hook personnalisé `useStore.js`.

```

1 import { create } from 'zustand';
2 import { jwtDecode } from 'jwt-decode';
3
4 const useStore = create((set) => ({
5   user: JSON.parse(localStorage.getItem('user')) || null,
6   isAuthenticated: !!localStorage.getItem('token'),
7
8   login: async (credentials) => {
9     const response = await authService.login(credentials);
10    const decoded = jwtDecode(response.data.access); // Extraction
        des claims
11    const userData = { id: decoded.user_id, username: decoded.
        username, role: decoded.role };
12
13    localStorage.setItem('token', response.data.access);
14    localStorage.setItem('user', JSON.stringify(userData));
15    set({ user: userData, isAuthenticated: true });
16  },
17
18  logout: () => {
19    localStorage.removeItem('token');
20    localStorage.removeItem('user');
21    set({ user: null, isAuthenticated: false });
22  }
23 }));

```

Listing 4.3 – Gestion globale de l'authentification avec Zustand

De plus, l'outil **Axios** a été configuré avec un intercepteur de requêtes. Ainsi, le jeton JWT est attaché de manière automatique et transparente dans l'en-tête `Authorization: Bearer <token>` de chaque requête sortante.

4.2.3 Conception Visuelle (UI/UX)

La Charte “Minimalist Blue”

Nous avons configuré Ant Design via un `ConfigProvider` pour écraser le style par défaut et injecter notre propre identité visuelle. La couleur *Navy* (`#0f172a`) sert de base pour la structure (Menu, Header), garantissant un fort contraste, tandis que la couleur d'accentuation *Teal* (`#14b8a6`) guide l'œil de l'utilisateur vers les actions prédictives.

Le Tableau de Bord Analytique

Le composant central de l'application est la `PredictionsPage`. Elle est organisée en plusieurs blocs logiques :

1. **Zone de Contrôle** : Un sélecteur stylisé en Navy foncé permettant de cibler un match.

2. **Hero Banner (Scorecard)** : Affiche les logos des équipes, leurs formes récentes sous forme de “badges” visuels (W, D, L) et donne la prédiction principale en très gros caractères.
3. **Radar Tactique** : Superpose la forme tactique (Possession, Passes, Tirs) de l'équipe locale et extérieure. Si la surface du polygone *Home* couvre entièrement le polygone *Away*, cela valide visuellement l'écart de niveau.
4. **Compositions (Lineups)** : Présente le schéma tactique (ex : 4-2-3-1) et dresse la liste des joueurs titulaires pour fournir le contexte matériel de la prédiction.

[Espace réservé pour la capture d'écran du Dashboard React]

FIGURE 4.1 – Aperçu de l'interface du Dashboard analytique

[Espace réservé pour la capture d'écran de l'authentification]

FIGURE 4.2 – Aperçu de l'interface de Connexion et Sécurité

4.3 Tests, Validation et Résultats

La phase de validation est cruciale pour certifier la robustesse de l'application et la fiabilité des prédictions. Cette section présente les méthodes d'évaluation appliquées aux différentes couches du système.

4.3.1 Évaluation du Modèle d'Intelligence Artificielle

Pour évaluer de manière empirique notre modèle **XGBoost**, nous avons divisé notre jeu de données historique en un ensemble d'entraînement (80%) et un ensemble de test (20%).

Matrice de Confusion Multiclasse

La matrice de confusion permet de visualiser les performances de l'algorithme sur les trois classes possibles : Victoire à Domicile (H), Match Nul (D), et Victoire à l'Extérieur (A).

		Prédictions du Modèle		
		Home (H)	Draw (D)	Away (A)
Réalité	Home (H)	Vrais Positifs (H)	Faux Négatifs	Faux Négatifs
	Draw (D)	Faux Positifs	Vrais Positifs (D)	Faux Positifs
	Away (A)	Faux Positifs	Faux Négatifs	Vrais Positifs (A)

TABLE 4.1 – Structure de la Matrice de Confusion pour l'évaluation du modèle

Métriques de Performance (Accuracy & F1-Score)

Suite à l'entraînement, nous avons obtenu une **Précision globale (Accuracy) de 62%**. Dans le domaine très spécifique des paris sportifs et de la prédiction de football, une précision supérieure à 55% est souvent le seuil de rentabilité (*break-even point*) pour les modèles algorithmiques, rendant notre résultat particulièrement satisfaisant.

De plus, grâce à l'application rigoureuse de l'algorithme **SMOTE** détaillé au Chapitre 2, le **Rappel (Recall)** pour la classe minoritaire "Match Nul" a considérablement augmenté. Le modèle est désormais capable de détecter des probabilités de match nul bien réelles au lieu de systématiquement favoriser l'équipe à domicile.

4.3.2 Validation du Backend (API REST)

Nous avons utilisé des outils standardisés tels que **Postman** et **pytest** pour valider de bout en bout les *endpoints* de notre API Django.

Scénarios de Tests d'Intégration

Voici un extrait des scénarios validés garantissant la sécurité et le bon fonctionnement de la plateforme :

Cas de Test	Résultat Attendu	Statut
Connexion avec des identifiants valides	Retourne HTTP 200 OK + Access/Refresh Tokens	Succès
Connexion avec un mot de passe erroné	Retourne HTTP 401 Unauthorized	Succès
Accès à une route protégée sans Token	Retourne HTTP 401 Unauthorized	Succès
Accès à l'administration sans rôle ADMIN	Retourne HTTP 403 Forbidden	Succès
Requête de prédiction sur un match valide	Retourne HTTP 200 + Probabilités H/D/A en JSON	Succès

TABLE 4.2 – Résultats des cas de tests d'intégration de l'API Backend

4.3.3 Validation du Frontend et de l'Ergonomie

L'interface utilisateur a subi des tests empiriques d'intégration et d'ergonomie (UI/UX) pour s'assurer que le livrable final est robuste.

- **Tests de Réactivité (Responsive Design)** : Vérification du bon affichage du Dashboard sur des résolutions *desktop* (1920x1080) et tablettes. Les graphiques dynamiques *Recharts* se redimensionnent correctement sans perte de qualité.
- **Gestion des Erreurs Utilisateur** : Si l'API retourne une erreur (par exemple, un token expiré) ou est momentanément indisponible, l'intercepteur Axios capte le code d'erreur et l'interface affiche un message convivial (via le composant **Alert** ou **Notification** d'Ant Design) au lieu de bloquer l'application.

4.4 Limites du système

Malgré les performances encourageantes, notre solution présente certaines limites techniques et fonctionnelles :

- **Dépendance aux API tierces** : La gratuité de l'API-Football limite le

volume de données récupérables en temps réel, ce qui peut restreindre l'actualisation immédiate de certains championnats mineurs.

- **Absence de données contextuelles** : Le modèle actuel ne prend pas encore en compte des facteurs externes imprévisibles tels que les conditions météorologiques, les transferts de dernière minute ou l'état psychologique individuel des joueurs.
- **Infrastructure de calcul** : L'utilisation d'une base SQLite limite le passage à l'échelle pour une application grand public supportant des milliers d'utilisateurs simultanés.

Conclusion du chapitre

Ce dernier chapitre confirme que le socle technique mis en place respecte les standards de qualité de l'ingénierie logicielle. L'implémentation complète, couplée à un modèle d'IA finement paramétré et rigoureusement testé, aboutit à un produit final stable, sécurisé et performant. Bien que des limites subsistent, elles ouvrent des perspectives d'évolution pour de futures versions de la plateforme.

Conclusion générale

Le projet “Intelligent Football Prediction Dashboard” a représenté un défi d’ingénierie complet, exigeant la maîtrise simultanée de trois domaines distincts : le génie logiciel (architecture N-Tiers, REST), la science des données (Machine Learning, XGBoost, SMOTE) et l’expérience utilisateur (React, Visualisation de données).

Nous avons réussi à concevoir un système automatisé capable non seulement de récolter des données mondiales via des API tierces, mais de les structurer pour en extraire une valeur prédictive. L’implémentation de fonctionnalités avancées, telles qu’un pipeline d’authentification robuste par JWT, une protection des ressources par rôles, et la synchronisation intelligente par lot (*Batch processing*) démontrent la maturité technique de la solution. L’interface utilisateur, s’appuyant sur les principes du design analytique, permet aujourd’hui à n’importe quel décideur ou passionné de comprendre instantanément les dynamiques d’un match complexe.

Bien que le football reste, par nature, sujet à des événements imprévisibles (cartons rouges précoces, conditions météorologiques), les probabilités mathématiques offrent une aide précieuse.

Perspectives d’Évolution

Plusieurs axes d’amélioration peuvent être envisagés pour accroître la précision et l’utilité du système :

- **Deep Learning et Réseaux de Neurones Récurents (RNN)** : Pour mieux capturer la temporalité des performances (Séquences de matchs) au lieu d’utiliser de simples moyennes lissées.
- **Données individuelles des joueurs** : Intégrer les blessures, les suspensions et l’historique personnel (ex : *Expected Goals* — xG individuels) directement dans le vecteur de caractéristiques du modèle, rendu désormais possible par notre nouvelle table **MatchLineup**.
- **Marchés financiers (*Odds*)** : Connecter l’application à des flux de données en temps réel pour superposer nos prédictions avec les cotes en direct (*Live Odds*) des bookmakers.

Ces évolutions permettraient de transformer ce projet académique robuste en une véritable plateforme d’aide à la décision professionnelle pour les clubs de football et

les analystes du monde sportif.

Bibliographie

- [1] M. J. Maher, “Modelling association football scores,” *Statistica Neerlandica*, vol. 36, no. 3, pp. 109–118, 1982.
- [2] M. J. Dixon and S. G. Coles, “Modelling association football scores and inefficiencies in the football betting market,” *Journal of the Royal Statistical Society : Series C (Applied Statistics)*, vol. 46, no. 2, pp. 265–280, 1997.
- [3] R. P. Bunker and F. Thabtah, “A machine learning framework for sport result prediction,” *Applied Computing and Informatics*, vol. 15, no. 1, pp. 27–33, 2019.
- [4] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [5] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] T. Chen and C. Guestrin, “Xgboost : A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, (San Francisco, CA, USA), pp. 785–794, 2016.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and P. W. Kegelmeyer, “Smote : Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [8] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” Tech. Rep. RFC 7519, Internet Engineering Task Force (IETF), 2015.